

Национальный исследовательский университет ИТМО
Факультет прикладных компьютерных наук
Факультет прикладных компьютерных наук

Новокшанов Евгений Андреевич

Поддержание базы автономных систем на основе данных протокола BGP

Магистерская диссертация

Допущена к защите.
Зав. кафедрой:
— —

Научный руководитель:
научный руководитель Белявцев И. П.

Рецензент:
рецензент —

Санкт-Петербург
2026

ITMO University
Faculty of Applied Computer Science
Faculty of Applied Computer Science

Evgeny Novokshanov

Maintaining the Autonomous Systems Database Based on BGP Protocol Data

Graduation Thesis

Admitted for defence.

Head of the chair:

--

Scientific supervisor:
supervisor Ivan Belyavtsev

Reviewer:

reviewer –

Saint Petersburg
2026

СОДЕРЖАНИЕ

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ	7
СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ	9
ВВЕДЕНИЕ	10
Актуальность темы исследования	10
Степень разработанности темы	11
Решаемая проблема	12
Объект и предмет исследования	12
Цель и задачи работы	12
Методы исследования	13
Методологическая и теоретическая основа	14
Научная новизна	14
Положения, выносимые на защиту	14
Степень достоверности и апробация результатов	15
Практическая значимость	15
Структура работы	16
1. Предметная область	17
1.1. Автономные системы и междоменная маршрутизация	17
1.2. BGP как базовый протокол междоменной маршрутизации	17
1.3. Multiprotocol extensions и поддержка нескольких семейств адресов	18
1.4. CIDR и longest prefix match как фундамент задачи	19
1.5. Route Refresh и повторная синхронизация состояния	19
1.6. BGP Monitoring Protocol и наблюдаемость состояния	20
2. Постановка проблемы	21
2.1. Нестабильность и пересылка избыточных обновлений	21
2.2. Теоретический взгляд: задача устойчивых путей	21
2.3. Роль публичных коллекторов маршрутов	22
2.4. Ограничения пакетной модели обновления	22

2.5.	Локальная динамика и flap damping	23
2.6.	Системные следствия для прикладного сервиса	24
3.	Обзор литературы	25
3.1.	PATRICIA и radix-подходы	25
3.2.	LC-trie	25
3.3.	Хеширование и поиск по длинам масок	26
3.4.	Controlled prefix expansion	26
3.5.	Tree Bitmap	27
3.6.	Poptrie	27
3.7.	Adaptive Radix Tree	28
3.8.	Сравнение по критериям	28
4.	Постановка задачи	30
4.1.	Гипотеза	30
4.2.	Функциональные требования	31
4.3.	Нефункциональные требования	31
4.4.	Критерии оценки результатов	32
5.	Архитектура разработанного микросервиса	33
5.1.	Общая архитектурная идея	33
5.2.	Модель взаимодействия с GoBGP	34
5.3.	Пользовательский gRPC API	35
5.4.	Два варианта внутреннего хранилища	35
5.5.	Эксплуатационные ограничения	36
5.6.	Вынесение логики в отдельный микросервис как основной архитектурный результат	37
6.	Алгоритмы начального снимка и потоковой актуализации	38
6.1.	Начальный снимок как этап инициализации	38
6.2.	Разрешение перекрывающихся префиксов	38
6.3.	Замена полного состояния	39
6.4.	Потоковая обработка изменений	39
6.5.	Пересинхронизация после ошибок	39

6.6. Модель состояния и корректность	40
7. Реализация шардированного ART-хранилища	42
7.1. Общая идея шардирования	42
7.2. Внутреннее представление ключа	43
7.3. Upsert и Delete	43
7.4. Lookup по IP-адресу	44
7.5. Dump полного состояния	45
7.6. Контракт prefixStore	45
7.7. Целевая операция и интерпретация результатов	45
7.8. Согласованность параллельного доступа	46
7.9. Поведение во время ресинхронизации	47
8. Методика и организация экспериментов	48
8.1. Цель экспериментальной части	48
8.2. Сравнимые варианты	48
8.3. Конфигурация стенда	49
8.4. Наборы данных и сценарии	49
8.5. Почему именно эти метрики достаточны	50
8.6. Ограничения экспериментальной методики	50
9. Результаты экспериментов и их интерпретация	52
9.1. Полная выгрузка	52
9.2. Инкрементальное обновление	52
9.3. Lookup по IP	53
9.4. Влияние числа шардов	54
9.5. Поведение IPv6	54
9.6. Смешанный набор на 3 млн записей	55
9.7. Сводный итог сравнения	56
9.8. Совокупная стоимость потокового профиля нагрузки	57
10. Практические ограничения и направления развития	60
10.1. Ограничения текущей реализации	60
10.2. Возможные доработки реализации	60

ЗАКЛЮЧЕНИЕ	62
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	64
СПИСОК ИЛЛЮСТРАТИВНОГО МАТЕРИАЛА	68

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

В настоящей работе используются следующие термины с соответствующими определениями.

Автономная система (AS) – совокупность сетевых префиксов, управляемых одним административным субъектом и анонсируемых наружу по единой маршрутной политике; идентифицируется числом ASN (Autonomous System Number).

BGP (Border Gateway Protocol) – протокол обмена маршрутной информацией между автономными системами, описанный в RFC 4271 [20].

BMP (BGP Monitoring Protocol) – протокол наблюдения за состоянием BGP-сеансов и содержимым таблиц маршрутизации [23].

CIDR (Classless Inter-Domain Routing) – модель бесклассовой адресации, в которой принадлежность адреса префиксу описывается парой «адрес/длина маски» [8].

Длиннейшее совпадение префикса (Longest Prefix Match, LPM) – задача поиска по IP-адресу префикса наибольшей длины, которому этот адрес принадлежит.

Полный снимок маршрутов (snapshot) – единомоментный набор действующих BGP-маршрутов, формирующих полное начальное состояние таблицы.

Поток обновлений (stream) – последовательность отдельных BGP-сообщений типа update/withdraw, изменяющих состояние таблицы после загрузки начального снимка.

Adaptive Radix Tree (ART) – адаптивное префиксное дерево с переменным размером узлов, описанное в работе Лейса и др. [14].

GoBGP – открытая реализация BGP-демона на языке Go, предоставляющая внешнее gRPC-API для управления и наблюдения.

gRPC – система удалённого вызова процедур поверх HTTP/2, используемая для взаимодействия микросервиса с GoBGP и с клиентами.

BIFIT MITIGATOR – продукт компании, обеспечивающий защиту от распределённых атак отказа в обслуживании.

BIFIT COLLECTOR – продукт для сбора и анализа сетевого трафика крупного оператора связи.

SLA (Service Level Agreement) – соглашение об уровне обслуживания, фиксирующее количественные требования к качеству работы сервиса.

СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ

API – application programming interface

ART – Adaptive Radix Tree

AS – автономная система

ASN – Autonomous System Number

BGP – Border Gateway Protocol

BMP – BGP Monitoring Protocol

CIDR – Classless Inter-Domain Routing

CPU – центральный процессор (central processing unit)

DDoS – distributed denial of service

gRPC – gRPC Remote Procedure Calls

IP – Internet Protocol

IPv4 / IPv6 – четвёртая и шестая версии IP

LPM – longest prefix match

MP-BGP – multiprotocol BGP

MRT – multi-threaded routing toolkit (формат дампов)

NLRI – network layer reachability information

RIB – routing information base

RIS – Routing Information Service (RIPE NCC)

SLA – service level agreement

ВКР – выпускная квалификационная работа

ВВЕДЕНИЕ

Актуальность темы исследования

Современная защита от DDoS-атак, сетевая аналитика и телеметрия операторских сетей всё чаще опираются на маршрутный контекст адресов. Главный элемент этого контекста – принадлежность IP-адреса автономной системе: по ASN формируются правила фильтрации, выполняется скоринг источников трафика, анализируются flow-записи и сопоставляются события с операторской топологией.

Результаты работы внедряются в инфраструктуру продуктов BIFIT MITIGATOR (защита от DDoS) и BIFIT COLLECTOR (анализ операторского трафика). Оба продукта должны опираться на одну и ту же таблицу *IP-префикс* $\rightarrow$ *ASN*, выведенную из полной таблицы маршрутизации BGP; внешние геолокационные справочники с редким обновлением не подходят для принятия решений в режиме реального времени. Устаревшее соответствие префиксов и ASN может приводить к ошибкам фильтрации, некорректной атрибуции источников атаки и расхождению трактовки одного и того же события между продуктами. До начала работы логика построения и обновления этой таблицы была встроена в BIFIT COLLECTOR, а внутреннее хранилище *ranger* обновлялось по таймеру полной заменой снимка: потоковое применение одиночных изменений к in-memory базе не поддерживалось, а BIFIT MITIGATOR не мог переиспользовать ту же реализацию без дублирования кода и данных.

Естественным первичным источником актуальных данных остаётся сам протокол BGP [20]. Чтобы использовать его в сервисе сопоставления префиксов и ASN, нужно получить начальный снимок RIB, непрерывно обрабатывать update-сообщения, поддерживать IPv4 и IPv6 и подобрать in-memory структуру, которая одновременно обеспечивает малое время LPM-запроса и позволяет применять инкрементальные изменения отдельных префиксных записей без полной перестройки таблицы.

В режимах, где задержка актуализации на уровне нескольких часов не

влияет на прикладное решение, могут применяться внешние справочники. Для высоконагруженных сервисов фильтрации и сетевой аналитики такой лаг неприемлем: минуты расхождения локального состояния с глобальной маршрутной таблицей повышают риск ложных срабатываний и пропусков трафика [13, 6]. Отсюда следуют требования к задержке актуализации на уровне единиц секунд и к инкрементальному обновлению in-memory состояния вместо периодической полной перестройки таблицы.

Степень разработанности темы

В мировой литературе вопросы динамики BGP и стабильности маршрутов изучаются с конца 1990-х. Работы Лабовица о маршрутной нестабильности и времени сходимости BGP [13, 6] важны для данной работы потому, что показывают: маршрутное состояние не является статичным справочником и должно рассматриваться как изменяемый объект. Формальная постановка задачи устойчивых путей у Гриффина и др. [9] задаёт теоретический язык для обсуждения сходимости и переходных состояний. Эмпирические источники RIPE RIS, RouteViews и RIPE Atlas [19, 21, 1] релевантны как практическая база для получения снимков и потоков маршрутных изменений.

Алгоритмическая сторона longest prefix match имеет столь же длинную историю. PATRICIA-деревья [15], LC-trie Нильссона и Карлссона [17], controlled prefix expansion Шринивасана [24], Tree Bitmap Итертона [7] и Poptrie Асаи [3] рассматриваются как основные семейства структур, с которыми необходимо сравнить выбранное хранилище. Их роль в работе состоит не в прямом заимствовании реализации, а в задании критериев сравнения: время LPM-запроса, стоимость обновления, расход памяти, стоимость полной выгрузки и сложность сопровождения. Adaptive Radix Tree предложен Лейсом и др. [14] для in-memory индексов СУБД; в данной работе он рассматривается как изменяемое хранилище префиксных ключей, а не как классический маршрутизаторный алгоритм LPM.

Архитектурная парадигма «начальный снимок + поток обновлений» проработана в литературе по системам обработки потоков [12, 5, 2, 11]. Эти источники вводятся для обоснования модели, в которой начальное состояние

строится из полного снимка, а последующие изменения применяются как упорядоченная последовательность событий. В данной работе эта модель адаптируется к BGP-производному состоянию и используется совместно с выделением функции сопоставления «префикс $\rightarrow$ ASN» в отдельный сервис с общим контрактом для нескольких продуктов.

Решаемая проблема

Для продуктов DDoS-защиты и сетевой аналитики нужна единая точка истины для пары *префикс $\rightarrow$ ASN*, доступная нескольким потребителям и обновляемая с задержкой порядка единиц секунд. Прежняя схема сочетала встроенность логики в один продукт, опрос по таймеру и полную замену таблицы в *ranger*: при таком устройстве нельзя получить ни общий сервис для BIFIT MITIGATOR и BIFIT COLLECTOR, ни потоковое in-memory обновление без перестройки всей таблицы. Проблема, решаемая в работе, состоит в выделении сопоставления в автономный микросервис с общим gRPC API и в обеспечении штатного режима «начальный снимок + поток обновлений» для in-memory состояния без отказа от прикладно приемлемого времени чтения.

Объект и предмет исследования

Объектом исследования являются программные средства поддержки актуального отображения сетевых префиксов в автономные системы по данным BGP.

Предметом исследования являются архитектурные решения по выделению функции сопоставления «префикс $\rightarrow$ ASN» в отдельный микросервис и методы потоковой актуализации его in-memory таблицы по данным BGP, включая выбор изменяемого внутреннего хранилища, поддерживающего LPM и инкрементальные правки.

Цель и задачи работы

Цель работы – разработать и исследовать микросервис, который поддерживает актуальную таблицу соответствия *IP-префикс $\rightarrow$ ASN* по данным BGP

для потребителей DDoS-защиты и сетевой аналитики в режиме реального времени.

Для достижения цели в работе решаются такие задачи:

1. изучить предметную область междоменной маршрутизации, свойства BGP и проблему свежести маршрутного состояния;
2. проанализировать известные подходы к LPM и структурам хранения префиксных таблиц;
3. сформулировать функциональные и нефункциональные требования к сервису, работающему по схеме «начальный снимок + поток обновлений»;
4. спроектировать архитектуру микросервиса и интерфейсы его взаимодействия с источником маршрутных данных и потребителями;
5. реализовать загрузку начального снимка и потоковую обработку обновлений с восстановлением после разрыва соединения;
6. реализовать шардированное in-memory хранилище на базе Adaptive Radix Tree с операциями поиска, вставки, удаления и полной выгрузки;
7. провести сравнительный эксперимент с прежним вариантом и оценить компромиссы между временем чтения, скоростью обновления и стоимостью полной выгрузки.

Методы исследования

В работе применяются методы системного анализа предметной области, сравнительного анализа алгоритмов и структур данных, эксперимент в форме воспроизводимых бенчмарков на одном и том же стенде, а также аналитическое моделирование совокупной CPU-нагрузки для двух режимов работы. Каждый эксперимент по несколько раз воспроизводится, а полученные показатели сводятся в таблицы с одинаковым набором столбцов для двух конкурирующих реализаций.

Методологическая и теоретическая основа исследования

Методологическую базу составляют действующие RFC по BGP и его расширениям [20, 16, 8, 4, 18, 23], а также фундаментальные работы по динамике BGP [13, 6, 9]. Теоретической основой алгоритмической части служат работы по префиксным деревьям и LPM [15, 17, 22, 10, 24, 7, 3], а также оригинальная статья по Adaptive Radix Tree [14]. Архитектурный шаблон «начальный снимок + поток обновлений» взят из практики систем обработки потоков и распределённых данных [12, 5, 2, 11].

Научная новизна

Научная новизна работы состоит не в новом сетевом протоколе, а в постановке и решении прикладной задачи инфраструктурного уровня для продуктовой линейки BIFIT. Во-первых, функция сопоставления *IP-префикс* → *ASN* вынесена из состава BIFIT COLLECTOR в автономный микросервис с единым gRPC контрактом для нескольких потребителей – это меняет границы ответственности и устраняет необходимость дублировать логику между MITIGATOR и COLLECTOR. Во-вторых, реализован переход от периодической полной замены таблицы к потоковому применению BGP-событий к in-memory базе по шаблону «начальный снимок + поток обновлений». В-третьих, для практической осуществимости потока выбрано и внедрено шардированное in-memory ядро на базе Adaptive Radix Tree [14]: структура обеспечивает инкрементальное изменение отдельных префиксных записей, LPM-запросы, полную выгрузку и рассылку подписчикам при эксплуатационно приемлемых затратах CPU, хотя в маршрутной литературе ART чаще обсуждается в контексте СУБД, а не BGP-состояния.

Положения, выносимые на защиту

1. Функция сопоставления «префикс → ASN» выделена в отдельный микросервис с общим gRPC API для нескольких продуктов; граница сервиса совпадает с границей предметной задачи, а не с границей одного из потребителей.

2. Архитектурный шаблон «начальный снимок + поток обновлений», адаптированный к источнику BGP-данных, обеспечивает потоковое поддержание in-memory таблицы без полной перестройки при каждом изменении маршрута.
3. Шардированное in-memory хранилище на основе Adaptive Radix Tree для пары «префикс → ASN» поддерживает LPM, точечные правки и полную выгрузку и служит технической основой для пункта 2.
4. Алгоритм восстановления состояния после разрыва с источником, сводящий пересинхронизацию к повторной загрузке снимка с последующим переходом в режим потока.
5. Эмпирическая оценка преимуществ потокового варианта по операции инкрементального обновления и аналитическая модель совокупной CPU-стоимости двух вариантов хранилища под профили нагрузки; численные пределы, при которых пакетная схема перестаёт быть эксплуатационно осуществимой.

Степень достоверности и апробация результатов

Достоверность результатов обеспечивается тем, что все эксперименты проведены на одном и том же стенде с фиксированной конфигурацией CPU, оперативной памяти и версии среды исполнения Go, а измерения выполнены стандартным механизмом `go test -bench` на нескольких размерах таблицы и нескольких профилях нагрузки. Конфигурация стенда и набор данных детально описаны в главе 8. Апробация результатов проведена в инфраструктуре компании БИФИТ: сервис включён в сборки продуктов BIFIT MITIGATOR и BIFIT COLLECTOR и используется как единый источник пары «префикс → ASN» для нескольких внутренних потребителей.

Практическая значимость

Практическая значимость определяется применимостью результата как инфраструктурного сервиса в системах сетевой аналитики, защиты трафика

и операторской отчётности, где важны:

- задержка актуализации таблицы маршрутов на уровне единиц секунд;
- единая точка обслуживания нескольких клиентов через общий API;
- согласованное и воспроизводимое состояние при перезапусках;
- малая CPU-стоимость инкрементального обновления отдельных префиксных записей;
- унификация источника данных и снижение стоимости поддержки нескольких продуктов компании.

Структура работы

Работа содержит введение, десять глав основной части, заключение, список использованных источников и список иллюстративного материала. Главы 1–2 посвящены теоретическим основам BGP и динамике междоменной маршрутизации. Глава 3 содержит обзор структур LPM и обоснование выбора in-методу хранилища префиксной таблицы. В главе 4 формулируются задача исследования, гипотеза, требования и критерии успеха. Главы 5–7 описывают архитектуру микросервиса, алгоритмы потоковой актуализации и реализацию шардированного ART-хранилища. Главы 8 и 9 посвящены методике и результатам сравнительного эксперимента. В главе 10 разобраны ограничения текущей реализации и направления развития.

1. Предметная область

1.1. Автономные системы и междоменная маршрутизация

Интернет представляет собой объединение большого числа независимых сетей, принадлежащих различным организациям: операторам связи, облачным платформам, корпорациям, университетам, государственным структурам и другим участникам. На уровне административного управления эти сети объединяются в автономные системы (Autonomous Systems, AS). Каждая автономная система управляется единым субъектом и реализует собственную политику маршрутизации.

Междоменная маршрутизация отличается от внутридоменной тем, что её цель не ограничивается поиском кратчайшего пути. Здесь ключевую роль играют политические и экономические ограничения: приоритет партнёрских связей, экспортные и импортные политики, ограничения по префиксам и соображения отказоустойчивости. Именно поэтому BGP, в отличие от типичных IGP-протоколов, является policy-aware протоколом и работает не как алгоритм минимизации расстояния, а как механизм обмена достижимостью с учётом локальных политик [20, 9].

1.2. BGP как базовый протокол междоменной маршрутизации

Согласно RFC 4271 [20], BGP-4 является междоменным протоколом маршрутизации, основная задача которого состоит в обмене информацией о достижимости сетевых префиксов между BGP-speaking системами. Маршрутная информация в BGP включает не только факт достижимости префикса, но и последовательность автономных систем (AS-path), через которые эта достижимость проходит. Это позволяет одновременно выявлять петли маршрутизации и принимать policy-based решения о выборе лучшего маршрута.

Для целей настоящей работы важны несколько свойств BGP.

Во-первых, BGP оперирует префиксами, а не отдельными адресами. Следовательно, задача сопоставления IP-адреса автономной системе в конечном

счёте сводится к *longest prefix match* по актуальному набору объявленных префиксов.

Во-вторых, BGP изначально поддерживает *classless routing* и CIDR [8]: маршруты различной длины маски могут сосуществовать, перекрываться и конкурировать за роль лучшего маршрута для конкретного адреса.

В-третьих, BGP представляет состояние маршрутизации не как статическую таблицу, а как непрерывно эволюционирующее множество объявлений (*announcements*) и *withdraw*-событий. Это означает, что программная система, желающая использовать BGP как источник истины, не может ограничиться редким чтением снимка без риска работы с устаревшим представлением.

1.3. Multiprotocol extensions и поддержка нескольких семейств адресов

RFC 4760 [16] вводит расширения BGP-4 для переноса информации о достижимости нескольких сетевых протоколов одновременно. Для этого используются пары $\langle AFI, SAFI \rangle$ (*Address Family Identifier* и *Subsequent Address Family Identifier*) и специальные атрибуты *MP_REACH_NLRI* и *MP_UNREACH_NLRI*. Для рассматриваемой задачи это означает, что прикладной сервис должен корректно работать как минимум с двумя основными случаями:

- IPv4 unicast ($AFI = 1, SAFI = 1$);
- IPv6 unicast ($AFI = 2, SAFI = 1$).

Различие между IPv4 и IPv6 важно не только на уровне протокола, но и на уровне внутреннего поиска. Для IPv4 максимальная длина префикса составляет 32 бита, для IPv6 – 128 бит. Соответственно, реализация *longest prefix match*, основанная на переборе возможных длин маски или на байтовой навигации по структуре данных, имеет принципиально различные характеристики на этих двух семействах адресов; разница проявляется как в стоимости отдельного LPM-запроса, так и в общем числе кандидатных длин маски, которые может потребоваться проверить.

1.4. CIDR и longest prefix match как фундамент задачи

RFC 4632 [8] закрепляет практику Classless Inter-Domain Routing и фиксирует переход от классовой адресации к префиксной как ключевой механизм ограничения роста глобального маршрутизационного состояния. В classless-модели значение имеет не «класс сети», а явная пара *префикс* и *длина маски*.

Для прикладного сервиса это означает следующее. Если для одного и того же адресного пространства одновременно объявлено несколько перекрывающихся префиксов, то принадлежность конкретного IP-адреса определяется по наиболее специфичному (longest) из них. Простого словаря «адрес → значение» здесь принципиально недостаточно. Внутреннее хранилище обязано либо нативно поддерживать LPM, либо предоставлять структуру, поверх которой LPM может быть эффективно реализован.

1.5. Route Refresh и повторная синхронизация состояния

RFC 2918 [4] описывает механизм Route Refresh Capability, позволяющий BGP-speaker запрашивать переобъявление соответствующего Adj-RIB-Out у соседа без полного перезапуска BGP-сессии. RFC 7313 [18] расширяет этот механизм, вводя маркеры начала и завершения refresh-последовательности (Beginning of RR, End of RR), что позволяет более корректно и менее разрушительно проверять и корректировать несогласованное состояние.

Для сервиса, работающего по схеме «снимок + поток обновлений», это особенно важно. Поточковая модель почти неизбежно сталкивается с вопросом ресинхронизации:

- что делать после разрыва соединения с источником данных;
- как убедиться, что между двумя моментами наблюдения не были потеряны withdraw-события;
- как безопасно вернуть состояние к согласованному виду без полной остановки и перезапуска всей системы.

Механизмы Route Refresh позволяют рассматривать такие сценарии не как ad-hoc инженерный костыль, а как легитимную часть протокольной модели BGP.

Дополнительный родственный сюжет – BGP route flap damping [25]. Хотя данная работа не реализует damping явно, его теоретический смысл, состоящий в подавлении нестабильности и сглаживании церемоний объявления/отзыва, важен для понимания того, почему обработка update-стрима не может быть наивной.

1.6. BGP Monitoring Protocol и наблюдаемость состояния

RFC 7854 [23] определяет BGP Monitoring Protocol (BMP) как специальный протокол для мониторинга BGP-сессий. В контексте настоящей работы BMP интересен не как непосредственно используемый транспорт, а как иллюстрация общей архитектурной идеи: маршрутизационное состояние имеет смысл собирать, наблюдать и передавать как отдельный поток данных, а не только как побочный эффект работы маршрутизатора.

Хотя в реализации, рассматриваемой в работе, интеграция строится не через BMP, а через GoBGP gRPC API, концептуально оба подхода принадлежат одному классу решений: отделение маршрутизационного plane от сервисов, использующих его как источник данных.

2. Постановка проблемы

2.1. Нестабильность и пересылка избыточных обновлений

Серия работ Лабовица и соавторов на рубеже 2000-х показала, что междоменная маршрутизация в реальной сети ведёт себя совсем не как статическая структура. В *Internet Routing Instability* [13] зафиксировано, что объём BGP-обновлений в среднем кратно превосходит интуитивный сценарий «редкие, осмысленные изменения». Существенная доля сообщений повторная, избыточная или возникает из-за патологий смежных маршрутизаторов. Любой прикладной сервис, привязанный к BGP-состоянию, обязан рассчитывать на непрерывные изменения как на штатный режим, а не как на аномалию.

В *Delayed Internet Routing Convergence* [6] та же картина дополнена временным срезом: после изменения топологии глобальная сходимость занимает минуты, а в худших сценариях – значительно больше. Для прикладной системы, опирающейся на отображение *префикс* $\rightarrow$ *ASN*, неопределённость возникает на двух уровнях:

1. собственно динамика BGP, заданная распределённой природой протокола и не поддающаяся одностороннему контролю;
2. дополнительная задержка локального обновления таблицы внутри самой прикладной системы.

Первое нельзя устранить – его остаётся только принять как свойство среды. Второе целиком определяется архитектурой сервиса, и именно его снижением занимается настоящая работа.

2.2. Теоретический взгляд: задача устойчивых путей

Гриффин, Шеферд и Вилфонг [9] формализовали работу BGP как Stable Paths Problem (SPP). Модель показывает, что протокол ищет локально устойчивый выбор маршрутов в присутствии политик, и этот процесс в

общем случае не обязан сходиться к единому глобальному оптимуму. Для прикладной части отсюда следуют два наблюдения.

Во-первых, у BGP нет «идеальной глобальной истины»: один и тот же префикс одновременно может выглядеть по-разному с точки зрения разных наблюдателей в зависимости от их положения в графе и от их собственных политик.

Во-вторых, если даже сам протокол достигает согласия только в пределе, то нет смысла строить прикладной сервис как редкий синхронный пересчёт. Гораздо ближе к реальности здесь модель, в которой состояние эволюционирует во времени и обновляется отдельными событиями.

2.3. Роль публичных коллекторов маршрутов

Платформы RIPE Routing Information Service [19] и RouteViews [21] многие годы служат основными источниками данных как для академических исследований BGP, так и для эксплуатационной аналитики. RIS опирается на распределённую сеть коллекторов, собирающих маршрутные данные у добровольных партнёров на крупных точках обмена трафиком и через multihop-сеансы. RouteViews предоставляет доступ к текущим и историческим данным коллекторов и к архивам в формате MRT.

Для настоящей работы важно, что именно эти платформы давно живут в двойной модели данных: периодические дампы таблиц плюс непрерывный поток обновлений. Иначе говоря, даже на исследовательском уровне схема «снимок + поток» считается естественным способом организации маршрутных данных. Хорошая иллюстрация – механизм RIS Beaconsing [1], в котором заранее известные префиксы объявляются и отзываются по расписанию, превращая BGP в контролируемый эксперимент.

2.4. Ограничения пакетной модели обновления

Допустим, прикладная система раз в несколько часов целиком перестраивает таблицу *префикс* $\rightarrow$ *ASN* по BGP полной таблице маршрутов. У такого подхода есть понятные преимущества: не нужно отслеживать инкременталь-

ные события, не нужно следить за порядком обновлений, а внутреннее хранилище можно оптимизировать под чтение и под редкую целикомую замену.

В динамической среде BGP такая схема упирается сразу в несколько ограничений:

1. между фактическим изменением маршрута и его появлением в локальной таблице возникает лаг; для систем реального времени лаг в часы – это уже работа с устаревшей картиной мира;
2. чем чаще приходится перестраивать таблицу, тем дороже это обходится: стоимость одной перестройки определяется размером таблицы n , а не объёмом фактических изменений;
3. локальная правка одного префикса требует работы, сопоставимой по объёму с полной сборкой таблицы заново;
4. с ростом числа маршрутов (сегодня IPv4 уже превысил миллион, а IPv6 – сотни тысяч префиксов) и частоты их изменений пакетная схема начинает не укладываться в требования по задержке и по потреблению CPU.

Отсюда естественный переход к другой схеме: начальный снимок нужен только при старте, а дальше актуальность таблицы поддерживается потоком обновлений.

2.5. Локальная динамика и flap damping

Динамика BGP не сводится к «полезным» изменениям топологии. Заметная часть update-трафика порождается локальными нестабильностями – короткоживущими событиями, повторными withdraw/announce одного и того же префикса, последствиями переключения на резервные пути. RFC 2439 [25] описывает классический механизм route flap damping, который подавляет излишне нестабильные маршруты экспоненциально убывающим штрафом.

Прикладному сервису flap damping не нужно реализовывать самостоятельно, но он остаётся важной иллюстрацией двух более общих наблюдений:

- поток BGP-обновлений несёт значительную долю избыточной информации;
- потоковая часть сервиса должна выдерживать всплески, многократно превышающие средний поток событий.

Это закладывает **мягкое, но конкретное требование** к структуре хранения: она обязана применять инкрементальные изменения отдельных префиксных записей без полной перестройки и без резкого роста CPU-затрат. Это требование вместе с семантикой Lookup определяет выбор алгоритмов в главах 4 и 7.

2.6. Системные следствия для прикладного сервиса

Соберу свойства, разобранные выше, в четыре положения о проектировании.

1. Модель «редкий пересчёт всего состояния» концептуально конфликтует с природой BGP: протокол не статичен, а непрерывно эволюционирует.
2. Рассинхронизация неизбежна; правильное проектирование закладывает дисциплину пересинхронизации как штатный режим, а не как обработку аварии.
3. **Стоимость одного события на стороне сервиса должна оставаться небольшой даже при высокой интенсивности потока, иначе потоковая модель теряет смысл.**
4. Семантика Lookup не должна зависеть от того, в какой момент произошло конкретное обновление: видимое клиенту состояние обязано соответствовать какой-то определённой точке последовательности применённых событий.

Эти четыре положения становятся техническими требованиями к сервису, формализованными в главе 4 как функциональные и нефункциональные ограничения.

3. Обзор литературы

Выбор структуры хранения для таблицы *префикс* $\rightarrow$ *ASN* связан не только с задачей *longest prefix match*. Для рассматриваемого микросервиса важны пять свойств: время точечного LPM-запроса, **стоимость** инкрементального обновления, расход памяти, **стоимость** полной выгрузки таблицы и сложность реализации в многопоточном *in-memory* сервисе. Поэтому далее рассматриваются не абстрактно лучшие LPM-алгоритмы, а семейства структур, релевантные для сочетания чтения, обновлений и выгрузки состояния.

3.1. PATRICIA и radix-подходы

Классическая работа Моррисона [15] вводит PATRICIA как сжатое префиксное дерево: в нём устраняются длинные цепочки узлов с единственным потомком. Для IP-префиксов эта идея важна потому, что адресное пространство разрежено, а обычное побитовое дерево быстро создаёт много служебных узлов.

PATRICIA и близкие radix-подходы естественно поддерживают префиксную семантику: поиск может идти по пути адреса с запоминанием последнего совпавшего префикса. Их сильная сторона – концептуальная близость к LPM и отсутствие необходимости перебирать все возможные длины масок. Однако для данной работы одной только LPM-семантики недостаточно. Необходимо также обеспечить конкурентные вставки и удаления, полную выгрузку таблицы и простой общий контракт для двух реализаций хранилища. Поэтому PATRICIA рассматривается как базовая точка сравнения, но не как непосредственно выбранная структура.

3.2. LC-trie

LC-trie Нильссона и Карлссона [17] развивает идею сжатого дерева и использует уровневую компрессию для уменьшения глубины поиска. Этот подход релевантен, поскольку он показывает, как можно снижать задержку LPM-запроса в программной реализации без перехода к специализированному оборудованию.

Ограничение LC-trie для рассматриваемой задачи связано с обновлениями. Структуры, оптимизированные под плотную и сжатую форму дерева, часто требуют более сложной перестройки при изменении префиксов. В маршрутизаторном сценарии, где основная нагрузка приходится на LPM-запросы, это приемлемо. В микросервисе, который должен применять поток BGP-событий по одному изменению, стоимость вставки и удаления становится критичной. Поэтому LC-trie фиксирует ориентир по времени чтения, но не закрывает требование к простой потоковой актуализации.

3.3. Хеширование и поиск по длинам масок

Хеш-таблица хорошо подходит для точного поиска по ключу: вставка, удаление и чтение в среднем имеют малую вычислительную стоимость. Для LPM, однако, ключом является не только адрес, но и длина маски. Один из практических вариантов состоит в хранении нормализованных префиксов в хеш-таблицах и проверке кандидатных масок от наиболее специфичной к менее специфичной. Подходы такого класса обсуждаются, в частности, в работе Вальдвогеля и соавторов [22].

Такой вариант релевантен для данной работы потому, что он хорошо согласуется с инкрементальными изменениями: конкретный префикс можно добавить или удалить без перестройки всей таблицы. Ограничение состоит в стоимости LPM-запроса, особенно для IPv6: при прямом переборе может проверяться до 129 длин масок. Кроме того, хеш-таблица сама по себе не даёт упорядоченной полной выгрузки, что важно для API полного снимка и сравнения состояний.

3.4. Controlled prefix expansion

Controlled prefix expansion, описанный Шринивасаном и соавторами [24], ускоряет поиск за счёт контролируемого расширения множества хранимых префиксов. Идея состоит в том, чтобы заменить часть поисковой работы заранее подготовленным представлением данных. Этот подход важен как пример явного обмена памяти на время LPM-запроса.

Для forwarding-plane устройств такой компромисс может быть оправдан: память выделяется заранее, а запросы чтения доминируют над обновлениями. Для сервиса потоковой актуализации ситуация иная. Дополнительный расход памяти и необходимость обновлять расширенное представление усложняют обработку BGP-событий. Поэтому controlled prefix expansion используется в обзоре как пример подхода, ориентированного на ускорение чтения, но не выбирается в качестве основы внутреннего хранилища.

3.5. Tree Bitmap

Tree Bitmap Итертона и соавторов [7] представляет префиксное дерево в виде компактных битовых карт. Подход ориентирован на быстрый LPM-запрос и эффективное размещение данных в памяти: битовые карты позволяют быстро определить наличие дочерних узлов и префиксов без обхода большого числа пустых позиций.

Tree Bitmap релевантен как сильный представитель структур, специально разработанных для маршрутного поиска. Его ограничение в контексте данной работы состоит в том, что основная инженерная задача не сводится к максимальному ускорению LPM-запроса. Сервис должен поддерживать инкрементальные изменения, полную выгрузку и понятную синхронизацию шардов. Эти требования делают специализированную структуру для чтения менее удобной, чем изменяемый индекс общего назначения.

3.6. Poptrie

Poptrie Асаи и Охары [3] показывает, что LPM можно существенно ускорить на CPU общего назначения при агрессивном учёте кэш-локальности, битовых векторов и инструкции подсчёта единичных битов. Эта работа важна для обзора потому, что задаёт современный ориентир для программной реализации высокоскоростного LPM-запроса.

В то же время Poptrie сильнее всего раскрывается в сценарии, где таблица маршрутов используется преимущественно для чтения. Для рассматриваемого микросервиса ключевой режим иной: после начальной загрузки таблица

должна обновляться потоком BGP-событий. Поэтому Portrie рассматривается как важная альтернатива для будущей оптимизации чтения, но не как базовая структура текущей реализации.

3.7. Adaptive Radix Tree

Adaptive Radix Tree, предложенный Лейсом, Кемпером и Нойманом [14], изначально относится к in-memory индексам СУБД. В ART внутренние узлы имеют адаптивный формат `Node4`, `Node16`, `Node48` и `Node256`; структура выбирает представление в зависимости от локальной плотности ветвления. Для данной работы эта идея важна не как готовый маршрутизаторный алгоритм, а как изменяемый байтовый индекс с относительно компактным хранением и поддержкой вставок, удалений и обхода.

ART не является оптимальным решением для всех LPM-сценариев. В текущей реализации поиск по адресу выполняется через проверку кандидатных префиксов разной длины маски, что увеличивает стоимость `Lookup`, особенно для IPv6. Тем не менее ART хорошо соответствует инженерной постановке работы: локальные изменения отдельных префиксов не требуют полной перестройки таблицы, дерево можно разделить на шарды, а полный обход остаётся реализуемым через обход всех деревьев.

3.8. Сравнение по критериям

Качественная сводка по рассмотренным структурам приведена в табл. 1. Оценки в таблице не являются универсальным ранжированием алгоритмов; они отражают применимость к целевому профилю данной работы: микросервис, поток BGP-обновлений, операции `Lookup`, `Upsert`, `Delete` и `Dump`.

Из сравнения следует, что выбор ART не означает утверждения о его превосходстве как LPM-алгоритма во всех условиях. Он выбран как практический компромисс для системы, где после начальной загрузки доминирует поток инкрементальных изменений, а операции чтения и полной выгрузки должны оставаться работоспособными и измеримо ограниченными по времени, памяти и числу аллокаций. Именно поэтому в дальнейших главах рас-

Таблица 1 – Качественное сравнение структур хранения по ключевым критериям

Структура	Lookup	Update	Delete	Dump	Memory	Сложность
Слайс / массив	низк.	низк.	низк.	высок.	ср.	низк.
Хеш-таблицы	ср.	высок.	высок.	низк.	ср.	ср.
PATRICIA / radix	высок.	ср.	ср.	ср.	ср.	ср.
LC-trie	высок.	низк.	низк.	ср.	ср.	высок.
Controlled expansion	высок.	низк.	низк.	ср.	низк.	высок.
Tree Bitmap	высок.	ср.	ср.	ср.	высок.	высок.
Poptrie	высок.	низк.	низк.	ср.	высок.	высок.
ART (шард.)	ср.	высок.	высок.	ср.	ср.	ср.

сматривается шардированная реализация ART и её экспериментальное сравнение с прежней реализацией на ranger.

4. Постановка задачи

Исходное инфраструктурное ограничение состоит в сочетании двух факторов. Во-первых, логика построения таблицы *IP-префикс* $\rightarrow$ *ASN* находилась внутри BIFIT COLLECTOR, тогда как BIFIT MITIGATOR использовал тот же тип данных для задач DDoS-защиты: фильтрации, анализа источников атаки и согласованной трактовки сетевых событий. Без отдельного сервиса это приводило бы к дублированию реализации и риску расхождения копий. Во-вторых, прежний вариант хранилища на *ranger* обновлялся по таймеру полной заменой снимка; одиночная вставка сводилась к перестройке всей структуры, поэтому поток BGP-событий нельзя было применять к in-memory базе по одному событию с приемлемой вычислительной стоимостью.

Сервис, рассматриваемый в работе, обслуживает несколько внутренних потребителей и должен отвечать на три типа запросов:

- к какой автономной системе относится конкретный IP-адрес;
- каков полный текущий снимок таблицы соответствия префиксов автономным системам;
- какие изменения в таблице произошли с момента подписки клиента.

Прежняя связка «встроенная логика + *ranger*» удовлетворяла части требований к чтению, но не позволяла ни разделить продукты по границе сервиса, ни сделать потоковое обновление базовым режимом при задержке актуализации на уровне единиц секунд.

4.1. Гипотеза

Опираясь на свойства BGP, рассмотренные в главе 2, и критерии выбора in-memory структуры, сформулированные в главе 3, гипотеза работы задаётся следующим образом:

если вынести функцию сопоставления IP-префикс $\rightarrow$ ASN в автономный микросервис с общим gRPC API, заменить пери-

*одическую полную перестройку таблицы на модель «начальный снимок + поток BGP-обновлений» и использовать изменяемое шардированное ART-хранилище, то можно снизить **CPU-стоимость** одиночного инкрементального обновления по сравнению с вариантом на *ranger*, сохранив прикладно приемлемое время *Lookup* и работоспособную полную выгрузку таблицы.*

Архитектурный шаблон «снимок + поток» – частный случай более общей модели потоковой обработки состояния, описанной у Акидау и соавторов [5]; он хорошо сочетается с лог-ориентированными платформами Kafka [12] и Flink [2] и согласуется с трактовкой состояния как функции последовательности событий поверх базового снимка [11].

4.2. Функциональные требования

1. Сервис должен уметь подключаться к источнику маршрутного состояния (узлу GoBGP) и считывать полный снимок IPv4 и IPv6 маршрутов.
2. Сервис должен уметь принимать поток BGP update-событий и отражать его во внутреннем состоянии.
3. Сервис должен предоставлять API для точечного Lookup по IP-адресу.
4. Сервис должен предоставлять API для получения полного снимка таблицы.
5. Сервис должен предоставлять API подписки на инкрементальные изменения.
6. Сервис должен корректно обрабатывать вставки, удаления и повторное построение состояния после ресинхронизации.

4.3. Нефункциональные требования

1. Одиночное обновление не должно требовать полной перестройки всей таблицы.

2. Чтение из таблицы должно оставаться достаточно быстрым для прикладного использования в DDoS-защите и сетевой аналитике.
3. Полная выгрузка может быть медленнее, чем в реализации, оптимизированной под пакетную полную замену, но должна оставаться работоспособной для целевого размера таблиц.
4. Архитектура должна позволять одновременно обслуживать нескольких клиентов через единый API.
5. Реализация должна быть достаточно прозрачной и сопровождаемой, включая возможность профилирования и наблюдения внутреннего состояния.

4.4. Критерии оценки результатов

Критерии, по которым результат работы должен признаваться успешным, вытекают из исследовательской гипотезы и требований:

1. **CPU-время** одиночного инкрементального обновления (Upsert/Delete) у шардированного ART меньше, чем у варианта на `ranger`, не менее чем на несколько порядков на целевых размерах таблиц.
2. Время точечного Lookup остаётся в диапазоне, пригодном для синхронного gRPC-запроса; при проигрыше `ranger` этот проигрыш должен быть явно измерен и объяснён.
3. Полная выгрузка Dump сохраняется как рабочая операция: измеряются время выполнения, объём выделенной памяти и число аллокаций.
4. Потребление памяти не заявляется как оптимизированная метрика без отдельного эксперимента; в рамках работы оно контролируется через показатели bytes/op и allocs/op для операций, где эти данные измерены.
5. Поточковая модель рассматривается как штатный режим работы, если совокупная CPU-стоимость при профилях нагрузки из табл. 10 остаётся существенно ниже варианта с полной перестройкой на `ranger`.

5. Архитектура разработанного микросервиса

5.1. Общая архитектурная идея

Центральный архитектурный результат – не столько выбор конкретного дерева, сколько выделение сопоставления «префикс $\rightarrow$ ASN» в отдельный сетевой процесс между источником BGP и продуктами компании. Сервис занимает промежуточное положение между узлом GoBGP и потребителями; последние подключаются по одному и тому же gRPC контракту, независимо от того, реализовано внутреннее хранилище на `ranger` или на шардированном ART. В качестве источника используется GoBGP с gRPC API; в качестве потребителей – BIFIT MITIGATOR и BIFIT COLLECTOR. Логически архитектура изображена на рис. 1.

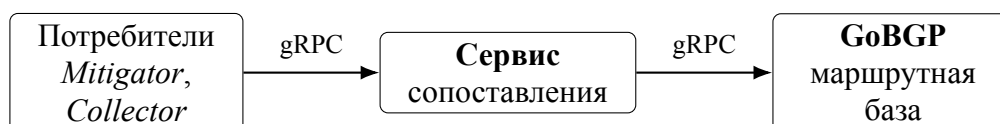


Рисунок 1 – Общая архитектура: сервис между потребителями и узлом GoBGP

На логическом уровне внутри сервиса можно выделить следующие компоненты:

- модуль конфигурации и запуска;
- gRPC-сервер пользовательского API;
- клиент GoBGP;
- доменный сервис синхронизации состояния;
- хранилище префиксов `prefixStore`;
- подсистема рассылки обновлений подписчикам;
- вспомогательные механизмы диагностики и профилирования (`pprof`).

Такое разделение важно, потому что оно изолирует два разных типа ответственности: получение и актуализацию маршрутизационного состояния

с одной стороны и обслуживание внешних запросов с другой. Внутренние логические слои показаны на рис. 2.

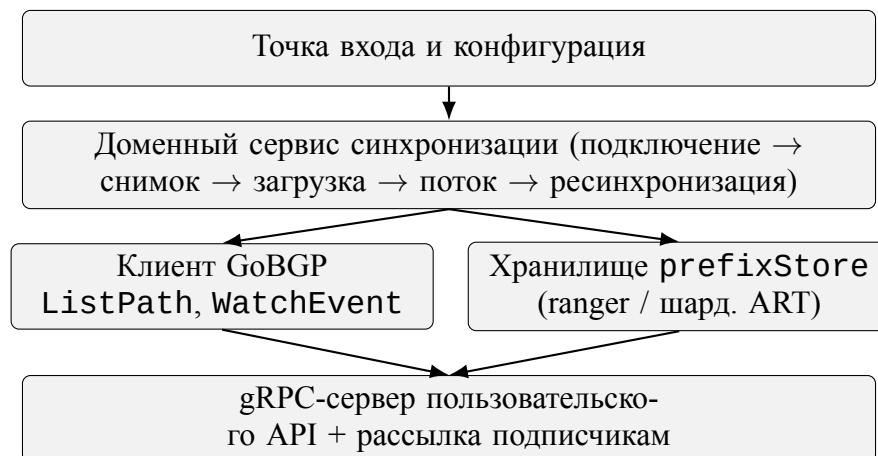


Рисунок 2 – Внутренние логические слои сервиса

5.2. Модель взаимодействия с GoBGP

Сервис взаимодействует с GoBGP в двух режимах: запросно-ответном и потоковом.

`ListPath` используется как запросно-ответная операция для получения полного снимка RIB. Она применяется при начальной загрузке, а также повторно при пересинхронизации после разрыва соединения.

`WatchEvent` открывает потоковое взаимодействие, через которое сервис получает объявления и отзывы маршрутов в том порядке, в котором их передаёт узел GoBGP.

Получается жизненный цикл такого вида:

1. подключиться к GoBGP;
2. прочитать полный снимок;
3. нормализовать и загрузить его во внутреннее хранилище;
4. перейти в режим обработки потока обновлений;
5. при ошибке или разрыве – переподключиться и пересинхронизироваться.

По смыслу это та же схема «начальная загрузка от снимка плюс непрерывный поток», что и в распределённых системах обработки событий [5, 2].

5.3. Пользовательский gRPC API

Для внешних потребителей реализуются три основных операции.

`GetASNInfo` выполняет LPM-запрос по IP-адресу и возвращает информацию о найденном ASN и наиболее специфичном префиксе.

`GetPrefixesASTable` возвращает полный снимок таблицы соответствия префиксов автономным системам.

`SubscribePrefixASUpdates` позволяет клиенту получать непрерывный поток изменений после момента подписки.

Такой набор операций покрывает как синхронные запросы, так и непрерывное наблюдение за изменениями. Семантика методов согласована с жизненным циклом, описанным в главе 6: клиент может получить полный снимок, затем подписаться на последующие изменения.

5.4. Два варианта внутреннего хранилища

В процессе работы сервиса поддерживаются два варианта внутреннего хранилища (см. рис. 3):

- **ranger** – прежнее решение, оптимизированное под полную замену таблицы;
- **шардированный ART** – новое решение, поддерживающее инкрементальные модификации.

Наличие двух вариантов полезно не только для постепенной миграции, но и для экспериментов: оно позволяет на одном и том же сервисном контуре измерять компромиссы между реализацией, оптимизированной под полную замену состояния, и реализацией, ориентированной на поток обновлений.

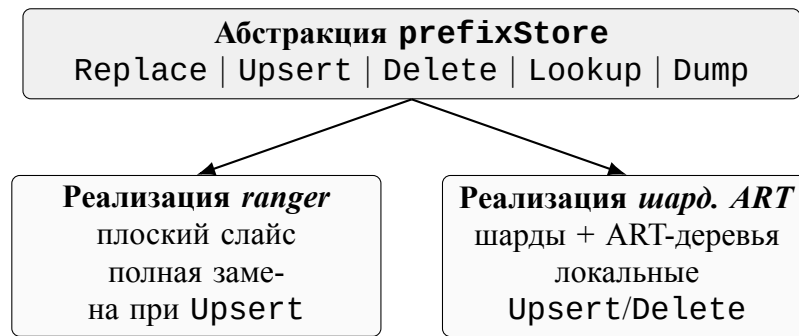


Рисунок 3 – Контракт `prefixStore` и две реализации

5.5. Эксплуатационные ограничения

При проектировании микросервиса с самого начала фиксируются несколько эксплуатационных ограничений, которые дальше определяют устройство внутренних механизмов.

- **Один источник маршрутных данных.** Сервис подключается к одному узлу GoBGP. Это сознательное упрощение: оно делает модель состояния однозначной и снимает вопрос о конфликтном слиянии данных от нескольких BGP-роутеров. Архитектура расширению не препятствует, но текущая версия работает по правилу «один источник – один поток событий – одно внутреннее состояние».
- **Одна копия состояния на всех клиентов.** Сервис держит одну логическую копию маршрутной таблицы. Альтернативная схема с копией на каждого клиента потребовала бы либо лишнего расхода памяти, либо более сложной сериализации обновлений; ни то ни другое для рассматриваемых сценариев не оправдано.
- **Гарантии видимости.** После того как обновление применилось к хранилищу, оно становится видимым последующим читателям и подписчикам. Строгой глобальной упорядоченности видимости для разных читателей не требуется – это согласуется с моделью BGP, где распределённое состояние сходится во времени.
- **Время отклика.** `Lookup` остаётся синхронным вызовом в пределах одного gRPC-запроса. Тяжёлые операции (`GetPrefixesASTable`,

подписка на поток) живут в отдельных RPC и не блокируют точечные запросы.

5.6. Вынесение логики в отдельный микросервис как основной архитектурный результат

До внедрения отдельного сервиса таблица «префикс → ASN» строилась и обновлялась внутри BIFIT COLLECTOR: второй продукт не мог опереться на ту же реализацию без копирования кода и состояния, а смена внутреннего хранилища потребовала бы правок в прикладной логике коллектора. После выделения функции в микросервис MITIGATOR и COLLECTOR становятся клиентами одного контракта: граница процесса совпадает с границей предметной задачи — сопоставление адресов и префиксов ASN по данным BGP.

Такое разделение даёт и эксплуатационный эффект. Обновление алгоритма загрузки снимка, политики нормализации префиксов или типа хранилища (*ranger* против шардированного ART) локализуется в одном месте; потребители продолжают вызывать те же RPC. Согласованность данных между продуктами перестаёт зависеть от дублирования кода; сопровождение не распадается на две независимые реализации одной и той же таблицы.

6. Алгоритмы начального снимка и потоковой актуализации

6.1. Начальный снимок как этап инициализации

При запуске сервис запрашивает полный снимок таблицы маршрутизации отдельно для IPv4 и IPv6. Такой разделённый подход упрощает логическую обработку, последующее распараллеливание и отдельную диагностику двух семейств адресов.

После получения маршрутов из GoBGP данные ещё не являются готовой таблицей *prefix* $\rightarrow$ *ASN*. Исходное множество маршрутов может содержать перекрывающиеся префиксы, а потребителям сервиса требуется представление, в котором для адресной области определён наблюдаемый ASN.

6.2. Разрешение перекрывающихся префиксов

Одной из важных задач при построении полной выгрузки является устранение неоднозначностей, связанных с перекрывающимися сетевыми префиксами. Если непосредственно скопировать все маршруты в хранилище, внешний пользователь при построении полной таблицы получит набор перекрывающихся диапазонов, неудобный для клиентских сценариев анализа.

Поэтому между этапом чтения маршрутов и этапом загрузки во внутреннее хранилище выполняется нормализация диапазонов: исходный набор маршрутов приводится к непересекающемуся множеству диапазонов, в котором каждой области сопоставляется **доминирующий** ASN. На уровне идеи это близко к обработке интервалов сканирующей прямой: события «начало диапазона» и «конец диапазона» обрабатываются в порядке адресов, а в каждой точке поддерживается множество активных префиксов с выбором наиболее специфичного из них.

Этот этап особенно важен для полной выгрузки: клиенты получают не произвольный внутренний набор префиксов, а нормализованное представление состояния.

6.3. Замена полного состояния

После построения снимка сервис выполняет операцию `Replace` для выбранного внутреннего хранилища. Для прежнего варианта на `ranger` это естественная операция, поскольку вся модель хранения основана на полной замене. Для ART-режима `Replace` тоже допустим, но используется только на этапе начальной инициализации и ресинхронизации, а не как нормальный способ обработки каждого `update`-события.

6.4. Поточковая обработка изменений

После начальной загрузки сервис подписывается на поток BGP-событий. Каждое событие интерпретируется как одна из двух базовых операций над `prefixStore`:

- `Upsert` маршрута;
- `Delete` маршрута.

В потоковой модели именно эти операции становятся главным рабочим режимом. Их обработка должна:

1. быть локальной, а не глобальной;
2. изменять только затронутый участок состояния;
3. сохранять корректность `Lookup`;
4. порождать уведомления для подписчиков через подсистему рассылки.

6.5. Пересинхронизация после ошибок

Потоковая модель имеет естественное ограничение: если соединение оборвалось, нельзя безусловно предположить, что локальное состояние осталось согласованным с источником. Поэтому система использует стратегию повторного подключения и ресинхронизации, изображённую как конечный автомат на рис. 4:

1. поток завершается или переходит в ошибку;
2. клиент закрывает соединение;
3. заново считывается полный снимок;
4. выполняется **Replace** для всего хранилища;
5. подписка на поток событий открывается повторно.

С точки зрения модели состояния это соответствует идее Route Refresh [4, 18] и подходу «снимок как начальная точка» в системах обработки потоков [5]: поток трактуется как эволюция состояния относительно базовой точки, и если эта точка стала недостоверной, она строится заново.

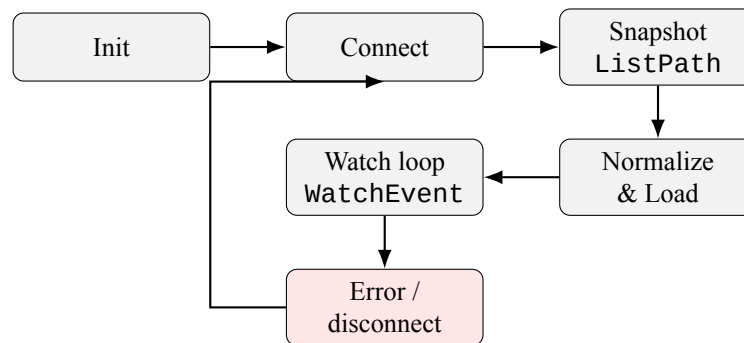


Рисунок 4 – Конечный автомат жизненного цикла повторного подключения и ресинхронизации

6.6. Модель состояния и корректность

Снимок и поток удобно рассматривать не как противоположности, а как две фазы жизни одного и того же материализованного состояния. Снимок нужен при старте, при пересинхронизации и при выгрузке полной таблицы наружу. Поток снижает задержку актуализации состояния, применяет инкрементальные изменения отдельных префиксных записей и передаёт обновления подписчикам.

Порядок применения событий задаётся самой сессией с GoBGP: сервис обрабатывает их в том порядке, в котором они приходят. По терминологии

работ по обработке потоков [5] это *processing-time*, а не *event-time*. Для прикладной задачи такой выбор обоснован по двум причинам. Во-первых, истинное время события у исходного BGP-роутера сервису недоступно – между ним и сервисом стоит GoBGP, агрегирующий поток. Во-вторых, важна не глобальная синхронизация по астрономическим часам, а локальная согласованность шагов: после применения события e_i и до применения e_{i+1} ответы на **Lookup** должны соответствовать состоянию после e_i . Эта гарантия обеспечивается дисциплиной работы с **prefixStore**: события обрабатываются по одному, а видимость их результата управляется блокировками внутри шардов.

Кратко состояние таблицы записывается как

$$S_n = f(S_0, e_1, e_2, \dots, e_n),$$

где S_0 – результат начальной загрузки, а $e_1, \dots, e_n$ – последовательно применённые BGP-события. Любой ответ **Lookup** клиента C в момент t_C соответствует состоянию после некоторого числа применённых событий S_k , $k \leq n(t_C)$. Полная выгрузка **GetPrefixesASTable** возвращает текущее представление состояния сервиса; в шардированной реализации её согласованность уточняется в главе 7. Подписка **SubscribePrefixASUpdates** даёт поток дельт, который позволяет клиенту, начав с S_k , восстановить $S_{k+1}, S_{k+2}, \dots$. Из этой записи видно главное: снимок и поток связаны единой семантикой, а **Replace** после ресинхронизации – переустановка базовой точки S_0 при сохранении того же типа функции f .

7. Реализация шардированного ART-хранилища

7.1. Общая идея шардирования

Одно из ограничений общей in-memory структуры заключается в конкуренции за блокировки. Если все операции чтения и записи проходят через одно дерево и одну блокировку, потоковая модель быстро сталкивается с конкуренцией за общий ресурс. Поэтому в работе применяется шардирование.

Вставка и удаление выполняются по ключу конкретного префикса. Этот ключ хешируется функцией FNV-32a, а значение хеша по модулю числа шардов определяет, в какой шард попадёт запись. Каждый шард содержит собственное ART-дерево и собственный `sync.RWMutex`. Такой подход локализует модификацию конкретного префикса: обновления разных префиксов, попавших в разные шарды, не конкурируют напрямую.

Важно, что шардирование не означает автоматической локализации Lookup по IP-адресу в одном шарде. Поскольку шард выбирается по ключу префикса, а не по исходному IP-адресу, разные кандидатные префиксы для одного адреса могут находиться в разных шардах. Например, короткий префикс и более специфичный префикс, покрывающие один и тот же адрес, не обязаны иметь одинаковый хеш. Поэтому поиск наиболее специфичного префикса описан отдельно в подразделе о Lookup.

Устройство шардированного хранилища изображено на рис. 5.

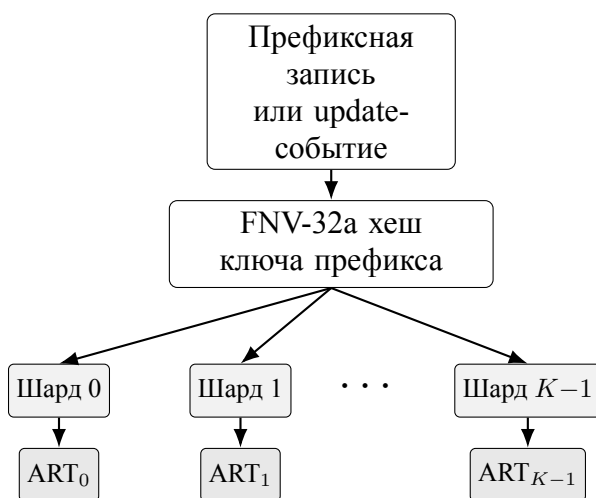


Рисунок 5 – Устройство шардированного ART-хранилища

7.2. Внутреннее представление ключа

ART-хранилище содержит записи префиксов, а не отдельные IP-адреса. Например, запись `8.0.0.0/5` -> `AS64512` хранится как одна логическая запись: адреса из диапазона `8.0.0.0–15.255.255.255` не разворачиваются в отдельные элементы и не копируются по нескольким шардам.

Перед записью или точным поиском префикс преобразуется в канонический байтовый ключ. Ключ включает три компонента:

1. маркер семейства адресов;
2. длину маски;
3. сетевой адрес после применения маски.

Для IPv4 ключ имеет длину 6 байт и формат

`[family, bits, ip0, ip1, ip2, ip3],`

где `family = 0x04`, `bits` – длина маски от 0 до 32, а оставшиеся байты задают нормализованный сетевой адрес. Например:

`8.0.0.0/5` → `[0x04, 5, 8, 0, 0, 0],`

`0.0.0.0/0` → `[0x04, 0, 0, 0, 0, 0].`

Для IPv6 используется тот же принцип, но ключ имеет длину 18 байт: маркер `0x06`, длина маски от 0 до 128 и 16 байт нормализованного сетевого адреса. Длина маски является частью ключа, поэтому `10.0.0.0/8`, `10.0.0.0/16` и `10.0.0.0/24` являются разными ключами, даже если начало адреса совпадает. Именно этот ключ далее хешируется функцией FNV-1a-32 для выбора шарда.

7.3. Upsert и Delete

Основное преимущество нового хранилища проявляется на операциях обновления. При `Upsert` сервис строит нормализованный ключ префикса,

выбирает по нему шард и локально модифицирует ART-дерево этого шарда. `Delete` обрабатывает `withdraw`-событие тем же способом: строится ключ удаляемого префикса, по нему выбирается шард, затем запись удаляется из соответствующего дерева.

В отличие от варианта на `ranger`, эти операции не требуют полной перестройки таблицы. Их стоимость зависит от длины ключа, размера выбранного шарда и локальной формы ART-дерева, а не от полного числа префиксов во всей таблице.

7.4. Lookup по IP-адресу

Точечный `Lookup` реализует `longest prefix match` по текущему состоянию хранилища. Важно, что при поиске не вычисляется шард исходного IP-адреса. Шардирование работает по ключам префиксов, поэтому поиск строится как последовательность точных проверок кандидатных префиксов.

Для входного IP-адреса сервис перебирает длины масок от наиболее специфичной к наиболее общей: для IPv4 это `/32`, `/31`, ..., `/0`; для IPv6 – `/128`, `/127`, ..., `/0`. На каждом шаге строится кандидатный префикс:

```
candidate = PrefixFrom(ip, bits).Masked().
```

Затем для него формируется канонический ключ, вычисляется

$$\text{shard} = \text{FNV1a32}(\text{key}) \bmod K,$$

и выполняется точный поиск этого ключа в ART-дереве выбранного шарда. Первый найденный кандидат возвращается как наиболее специфичный совпадающий префикс.

Такой порядок сохраняет корректность для широких префиксов. Если в хранилище есть запись `8.0.0.0/5`, а клиент выполняет `Lookup` для адреса `10.1.2.3`, то при переборе масок алгоритм дойдёт до длины `/5` и построит кандидат `8.0.0.0/5`. Для него получится тот же ключ `[0x04, 5, 8, 0, 0, 0]`, а значит и тот же шард, в который эта запись была вставлена. Поэтому широкие префиксы `/4`, `/5` или `/0` не нужно дублировать по нескольким шардам: они остаются одиночными префиксными записями.

Следствие этой схемы состоит в том, что один **Lookup** в общем случае может обращаться к разным шардам: разные кандидатные префиксы для одного IP-адреса имеют разные ключи и могут иметь разные значения хеша. В худшем случае проверяется 33 кандидата для IPv4 и 129 кандидатов для IPv6. Этим объясняется более высокая стоимость **Lookup** для IPv6 в экспериментальных результатах главы 9.

7.5. **Dump** полного состояния

Полная выгрузка в ART-режиме реализуется как обход всех шардов и сборка результатов. Это делает **Dump** дороже, чем в структуре, которая держит уже готовый линейный массив записей. ART-подход платит за изменяемость дополнительными затратами на обход, аллокации и опциональную сортировку. В реализации сортировка управляется параметром **ASMAP_STORE_ART_DUMP_SORT**; если порядок записей не важен, отключение сортировки уменьшает время полной выгрузки на больших таблицах.

Эта особенность не является ошибкой реализации. Она отражает компромисс: структура, оптимизированная под частые инкрементальные обновления, редко оказывается одновременно лучшей и для операции «отдать всё содержимое».

7.6. Контракт **prefixStore**

С точки зрения исследования важен явный контракт **prefixStore**, общий для двух вариантов хранилища. Контракт задан минимальным набором операций, описанных в табл. 2; это позволяет сопоставлять две реализации одного интерфейса, а не две несвязанные программы.

7.7. Целевая операция и интерпретация результатов

Оценивать новое хранилище только по времени **Dump** или по времени точечного **Lookup** недостаточно. Целевая функция работы – сделать потоковую актуализацию практически применимой. Поэтому главными измеряемыми параметрами являются CPU-время **Upsert** и **Delete**; **Dump** и

Таблица 2 – Контракт `prefixStore` и принципиальное поведение реализаций

Операция	Реализация <i>ranger</i>	Реализация <i>шард. ART</i>
<code>Replace</code>	полная замена слайса	параллельная загрузка по шардам
<code>Upsert</code>	пересборка всего слайса и полная замена ($O(n)$)	локальная модификация одного шарда; стоимость зависит от размера выбранного шарда и длины ключа
<code>Delete</code>	пересборка всего слайса	локальное удаление в одном шарде
<code>Lookup</code>	longest prefix match по линейной структуре	перебор кандидатных масок; для каждого кандидата поиск в шарде, выбранном по ключу префикса
<code>Dump</code>	копирование готового слайса (1 аллокация)	обход всех K шардов, опционально сортировка

`Lookup` фиксируют стоимость, которую приходится заплатить за изменяемость структуры. Эта установка используется при интерпретации результатов в главе 9.

7.8. Согласованность параллельного доступа

Шардирование само по себе не определяет модель согласованности; оно лишь снижает конкуренцию за блокировки между независимыми операциями. Чтобы `Lookup` и `Upsert` безопасно сосуществовали, в каждом шарде используется `sync.RWMutex`. Все операции записи (`Upsert`, `Delete`, локальная фаза `Replace`) удерживают блокировку на запись соответствующего шарда. `Lookup` при проверке очередного кандидатного префикса удерживает блокировку на чтение того шарда, куда попадает этот кандидат.

Такая дисциплина даёт следующие гарантии:

- внутри одного шарда параллельные операции чтения выполняются без взаимного блокирования;
- модификация одного шарда не блокирует операции в других шардах;
- `Lookup` может последовательно обращаться к нескольким шардам, но

каждое отдельное обращение согласовано с локальными изменениями соответствующего шарда;

- **Dump** реализуется обходом всех шардов с локальными блокировками на чтение.

Согласованность **Dump** в этой модели является шардовой, а не глобальной атомарной согласованностью всей таблицы: каждое значение, попавшее в выгрузку, корректно относительно своего шарда в момент чтения. Для прикладного сценария получения полной таблицы это рассматривается как текущее представление сервиса. Если клиенту требуется строгое восстановление состояния во времени, он должен использовать сценарий «полный снимок + последующая подписка на изменения», описанный в главе 6.

7.9. Поведение во время ресинхронизации

Отдельный режим работы – ресинхронизация. В этот момент сервис должен:

1. получить новый снимок;
2. подготовить из него корректное внутреннее представление;
3. заменить текущее состояние хранилища на новое;
4. возобновить обработку потока обновлений.

В шардированной реализации эта последовательность выполняется как скоординированная замена содержимого всех шардов. Параллельный **Lookup**, попадающий в момент перестроения, либо обслуживается из состояния соответствующего шарда до замены, либо ожидает завершения локальной фазы записи. Кратковременные различия видимости между шардами в момент ресинхронизации являются следствием выбранной модели и допустимы, поскольку ресинхронизация происходит только после сбоя или длительных пауз, а затем сервис снова переходит к штатному потоку обновлений.

8. Методика и организация экспериментов

8.1. Цель экспериментальной части

Экспериментальная часть должна ответить на вопрос, подтверждается ли выдвинутая гипотеза — в части возможности потокового обновления `in-memory` базы после выделения микросервиса. Для этого нужно измерить, как новое и прежнее внутренние хранилища ведут себя на наборе операций, соответствующих реальным сценариям использования сервиса:

- полная выгрузка таблицы (`Dump`);
- одиночное обновление маршрута (`Upsert`);
- точечный поиск по IP-адресу (`Lookup`);
- влияние числа шардов на ключевые операции;
- различия между IPv4 и IPv6.

8.2. Сравнимые варианты

В опытах сравниваются:

- **ranger** — прежний вариант хранилища на базе библиотеки `ranger`;
- **шард. ART** — новый вариант хранилища на базе шардированного `Adaptive Radix Tree`.

Это важный методический момент: сравнение выполняется не между двумя абстрактными структурами данных из учебника, а между двумя реальными вариантами внутреннего хранилища одного и того же сервиса с одним и тем же `prefixStore`-контрактом. Все различия, фиксируемые в эксперименте, можно интерпретировать как различия между реализацией, ориентированной на полную замену, и реализацией, ориентированной на поток обновлений, в рамках этого контракта.

8.3. Конфигурация стенда

Параметры экспериментального стенда приведены в табл. 3. Все измерения проводились на одной и той же физической машине, без использования контейнеров и виртуализации, с зафиксированным набором фоновых процессов.

Таблица 3 – Конфигурация стенда экспериментов

Параметр	Значение
CPU	AMD Ryzen AI 9 HX 370 (24 потока)
ОЗУ	32 ГБ LPDDR5X
Среда	Linux x86_64
Среда исполнения	Go 1.24+
Инструмент	go test -bench с теговой подсистемой bench, benchheavy
ART-параметры	16 шардов, WithDumpSort(false) в сценариях с большими n
Размеры таблиц	$n \in \{1\,000; 10\,000; 100\,000\}$; отдельный тяжёлый сценарий – 3 000 000 смешанных IPv4/IPv6 записей
Семейства адресов	IPv4 /24 (искусственные диапазоны 10.x.y.0/24, 11.x.y.0/24), IPv6 /64 в 2001:db8::/32; для тяжёлого сценария – смесь IPv4 /24 и IPv6 /64
Параметры запуска	count=2, benchtime=400ms/500ms

8.4. Наборы данных и сценарии

Для измерений используются синтетические наборы префиксов, а также сценарии, имитирующие реалистичную нагрузку. В частности:

- таблицы различного размера, вплоть до 10^5 префиксов в основных сериях и отдельный тяжёлый сценарий на $3 \cdot 10^6$ смешанных записей;
- отдельные наборы для IPv4 и IPv6;
- сценарии полного Dump;
- сценарии одиночного Upsert поверх уже загруженной таблицы;
- сценарии Lookup;

- sweep по числу шардов $K \in \{1, 8, 16, 32, 64\}$.

Такой дизайн эксперимента позволяет оценить не только «среднюю температуру», но и понять, какие операции становятся bottleneck для каждой реализации.

8.5. Почему именно эти метрики достаточны

Для данной работы важно подчеркнуть, что метрики выбираются не случайно, а отражают конкретные эксплуатационные сценарии.

Операция `Dump` соответствует сценарию получения полного снимка внешним потребителем, а также внутренним этапам начальной инициализации и ресинхронизации.

Операция `Upsert` соответствует основной сущности потокового режима: локальному отражению одного BGP-изменения. Именно её стоимость определяет, осмыслен ли переход от полной пересборки таблицы к потоку; измерение `Upsert` – прямая проверка этой части гипотезы, а не «бенчмарк ради ART».

Операция `Lookup` задаёт дополнительный разрез: приемлемость задержки точечного запроса при выбранном хранилище (включая реализацию LPM).

Измерение при разных значениях числа шардов показывает, где проходит граница между полезной параллельностью и издержками чрезмерного дробления состояния.

Именно эта совокупность метрик позволяет проверить, достигнута ли цель перехода к потоковой архитектуре, и подтвердить или опровергнуть выдвинутую гипотезу.

8.6. Ограничения экспериментальной методики

Экспериментальная часть измеряет прежде всего внутренние операции хранилища: полную загрузку, точечный поиск, инкрементальное обновление и полную выгрузку таблицы. Такой выбор позволяет изолировать различия

между двумя реализациями `prefixStore`, но оставляет за пределами измерений ряд факторов эксплуатации:

- сетевые задержки в внешних gRPC-вызовах;
- долгосрочную динамику при реальном потоке BGP update-событий за большие интервалы времени;
- поведение под многоклиентской нагрузкой на пользовательский API;
- влияние сборщика мусора и давления на аллокатор при длительной работе сервиса;
- воспроизведение реалистичного распределения длин префиксов из промышленных BGP-таблиц.

Для цели данной работы такое ограничение методики приемлемо: измерения проверяют именно тот слой, в котором различаются `ranger` и шардированный ART, и позволяют сравнить два варианта хранилища по ключевым операциям. Ограничения интерпретации результатов перечислены выше и учитываются в выводах главы 9.

9. Результаты экспериментов и их интерпретация

9.1. Полная выгрузка

Измерения показывают, что прежний вариант хранилища, основанный на `ranger`, значительно быстрее выполняет полную выгрузку таблицы. Это объясняется тем, что он концептуально хранит уже подготовленный линейный набор записей и может отдавать его почти как копию готового массива. Шардированный ART, напротив, должен обойти все шарды, декодировать ключи, собрать результат и (по умолчанию) отсортировать его.

Конкретные численные результаты приведены в табл. 4. На малых таблицах ($n = 1000$, $n = 10000$) разница составляет два–три порядка, а на $n = 100000$ при выключенной сортировке разрыв сокращается, оставаясь, однако, значительным.

Таблица 4 – Бенчмарк `Dump` для IPv4 /24

Бэкенд (n)	Время	Память	Аллокаций
<code>ranger</code> , $n = 1000$	$\approx 2,6$ мкс/оп	8 KiB	1
<code>ranger</code> , $n = 10\,000$	≈ 30 мкс/оп	80 KiB	1
<code>ranger</code> , $n = 100\,000$	$\approx 245\text{--}275$ мкс/оп	784 KiB	1
ART, $n = 1000$	≈ 337 мкс/оп	≈ 270 KiB	≈ 3970
ART, $n = 10\,000$	$\approx 3,2$ мс/оп	$\approx 3,0$ MiB	$\approx 35\,000$
ART, $n = 100\,000^*$	$\approx 13,4\text{--}13,8$ мс/оп	$\approx 33,4$ MiB	$\approx 351\,000$

* ART, $n = 100\,000$ – режим без сортировки дампа (`WithDumpSort(false)`).

Эти данные фиксируют существенный проигрыш ART по полной выгрузке: при $n = 10^5$ операция занимает 13–15 мс против сотен микросекунд у `ranger`. Для целевого профиля нагрузки это не опровергает гипотезу, поскольку `Dump` не является основным рабочим режимом потоковой модели, но задаёт ограничение для сценариев частых полных выгрузок.

9.2. Инкрементальное обновление

С точки зрения постановки задачи этот раздел – центральный: он показывает, что потоковое применение одиночных BGP-изменений к in-memory базе технически осуществимо, а не только объявлено в архитектуре. Именно на операции одиночного `Upsert` новый подход демонстрирует решающее

преимущество (табл. 5). У `ranger` каждый `Upsert` пересобирает весь список и вызывает `Replace`, что приводит к полной перестройке внутренней структуры. У ART та же операция выполняется локально в одном шарде, без вмешательства в остальные.

Таблица 5 – Бенчмарк одиночного `Upsert` поверх заполненной таблицы (IPv4)

n	<code>ranger</code>	ART (16 шардов)
1000	$\approx 3,0$ мс/оп	≈ 128 нс/оп
10 000	≈ 43 мс/оп	≈ 125 нс/оп
100 000	$\approx 587\text{--}616$ мс/оп	$\approx 165\text{--}169$ нс/оп

Разница достигает примерно **шести порядков** при $n = 10^5$: с ~ 600 мс/оп до ~ 170 нс/оп. Это главный эмпирический аргумент в пользу потоковой модели: без такой стоимости `Upsert` поддержание таблицы по одному событию не имело бы эксплуатационного смысла. Новая архитектура качественно меняет допустимую модель эксплуатации: если при `ranger` поток BGP-изменений практически несовместим с локальным обновлением, то при шардированном ART инкремент становится естественным базовым режимом – уже внутри выделенного микросервиса с общим API.

9.3. Lookup по IP

Это вспомогательный разрез относительно центральной цели работы (микросервис и поток обновлений), но он фиксирует цену точечного чтения после выбора структуры. По операции `Lookup` картина оказывается более неоднородной. Для IPv4 прежний вариант хранилища показывает меньшую задержку, а ART остаётся в зоне приемлемых, но более высоких значений. Для IPv6 разрыв становится ещё заметнее: текущая реализация ART выполняет `Lookup` существенно медленнее (табл. 6). Объяснение в том, что ART при поиске перебирает все возможные длины масок – до 33 для IPv4 и до 129 для IPv6.

Это важное ограничение текущей реализации. Оно показывает, что шардированный ART не является универсальным победителем по всем метрикам. Однако результат не опровергает гипотезу работы: цель состояла не

Таблица 6 – Бенчмарк Lookup (IPv4 /32 и IPv6 /128, $n \approx 10^5$)

Сценарий	ranger	ART (16 шардов)
IPv4 /32	151 нс/оп	400–500 нс/оп
IPv6 /128	168–178 нс/оп	3700–4000 нс/оп

в максимизации одной операции **Lookup**, а в достижении баланса между чтением и инкрементальной актуализацией. Возможные пути оптимизации **Lookup** для ART рассматриваются в главе 10.

9.4. Влияние числа шардов

Эксперименты показывают, что увеличение числа шардов не даёт линейного ускорения всех операций. Для **Lookup** выигрыш часто невелик или находится в пределах шума. Для **Dump** слишком большое число шардов может даже ухудшать результат из-за роста служебных издержек (табл. 7). Для update-сценариев умеренное шардирование оказывается полезным, потому что снижает конкуренцию между независимыми модификациями.

Таблица 7 – Измерение при разном числе шардов: **Dump** ART, IPv4, без сортировки

K (шардов)	$n = 5000$	$n = 100\,000$
1	–	$\approx 12,2\text{--}12,6$ мс/оп
8	$\approx 1,45\text{--}1,47$ мс/оп	$\approx 10,9\text{--}11,6$ мс/оп
16	$\approx 1,50\text{--}1,56$ мс/оп	$\approx 10,9\text{--}11,6$ мс/оп
32	$\approx 1,58\text{--}1,59$ мс/оп	$\approx 13,9\text{--}15,3$ мс/оп
64	$\approx 1,76\text{--}1,78$ мс/оп	$\approx 13,9\text{--}14,8$ мс/оп

Следовательно, число шардов является параметром настройки, а не правилом «чем больше, тем лучше». Практически разумной выглядит зона умеренного шардирования ($K \in [8; 32]$), а не экстремальные значения.

9.5. Поведение IPv6

IPv6 заслуживает отдельного внимания. По **Dump** разрыв между ranger и ART сохраняется на том же уровне, что и для IPv4: примерно 200–250 мкс

против 12,5–13,7 мс. По **Upsert** картина та же: ~ 889 мс у **ranger** против ~ 173–182 нс у **ART**, что снова даёт разницу более чем в пять порядков. По **Lookup** **ranger** быстрее в десятки раз, что объясняется значительно большим числом перебираемых длин масок в текущей реализации **ART**. Этот результат важен как ограничение реализации, а не как свойство самого подхода.

9.6. Смешанный набор на 3 млн записей

Для проверки масштабирования был отдельно выполнен тяжёлый прогон на 3 000 000 префиксных записей в смешанном режиме: IPv4 /24 и IPv6 /64. **ART** использовался с 16 шардами и выключенной сортировкой полной выгрузки. Результаты приведены в табл. 8.

Таблица 8 – Сравнение **ranger** и шардированного **ART** на смешанном наборе из $3 \cdot 10^6$ записей

Сценарий	ranger	ART (16 шардов, без сортировки)	Победитель
Replace / полная загрузка	27,423 с/оп	0,622 с/оп	ART
Lookup	185,3 нс/оп	2,899 мкс/оп	ranger
Dump	7,522 мс/оп	326,957 мс/оп	ranger
Upsert	28,178 с/оп	156,8 нс/оп	ART

Та же серия измерений показывает существенные различия по памяти и числу аллокаций. При **Replace** прежний вариант хранилища выделяет около 7,39 ГБ/оп и выполняет 357,5 млн аллокаций, тогда как **ART** – около 556 МБ/оп и 18,6 млн аллокаций. При **Upsert** разрыв ещё больше: около 7,56 ГБ/оп и 366,5 млн аллокаций у **ranger** против 44 В/оп и 3 аллокаций у **ART**. На **Lookup** и **Dump** ситуация обратная: **ranger** требует 30 В/оп и 2 аллокации для **Lookup**, а **ART** – около 1165 В/оп и 53 аллокации; для **Dump** значения составляют около 24 МБ/оп и 1 аллокации у **ranger** против 909 МБ/оп и 9,38 млн аллокаций у **ART**.

Этот прогон уточняет общую картину. На большой смешанной таблице **ART** заметно выигрывает не только в инкрементальном обновлении, но и в полной загрузке состояния: **Replace** быстрее примерно в 44 раза. При этом **Lookup** и **Dump** остаются слабыми местами **ART**: на смешанном наборе **Lookup** медленнее примерно в 15–16 раз, а **Dump** – примерно в 43 раза.

Следовательно, выбор хранилища зависит от профиля нагрузки: для состояния, обновляемого потоком BGP-событий, критичен **Upsert**, тогда как для почти неизменяемой таблицы с частыми чтениями преимущество остаётся у **ranger**.

9.7. Сводный итог сравнения

Сводка по сценариям приведена в табл. 9.

Таблица 9 – Итоговая сводка: какой вариант хранилища выигрывает в каком сценарии

Сценарий	Победитель	Комментарий
Полная выгрузка	ranger	готовый линейный массив; ART требует обхода дерева и дополнительных аллокаций
Одиночный Upsert	ART	5–6 порядков преимущества; решающий фактор для потокового режима
Lookup IPv4	ranger	разница в 2–3 раза; для прикладного сервиса допустимо
Lookup IPv6	ranger	разрыв в десятки раз; ограничение текущей реализации LPM
Полная загрузка Replace на $3 \cdot 10^6$ записей	ART	0,622 с против 27,423 с у ranger
Потоковое обновление	ART	отдельное BGP-событие применяется без полной перестройки состояния
Потоковое обновление на $3 \cdot 10^6$ записей	ART	Upsert: 156,8 нс/оп против 28,178 с/оп у ranger

Из сводки следуют четыре вывода, непосредственно связанные с измеренными операциями:

1. **ranger** сохраняет преимущество в сценариях, где состояние редко изменяется, а основная нагрузка приходится на чтение: на смешанном наборе $3 \cdot 10^6$ записей Lookup у **ranger** занимает 185,3 нс/оп против 2,899 мкс/оп у ART, а Dump – 7,522 мс/оп против 326,957 мс/оп;

2. ART лучше соответствует сценарию потокового применения BGP-событий: одиночный **Upsert** на смешанном наборе выполняется за 156,8 нс/оп, тогда как для **ranger** та же операция требует полной перестройки состояния и занимает 28,178 с/оп;
3. на полной загрузке большого смешанного набора ART также показывает меньшую стоимость: **Replace** занимает 0,622 с/оп против 27,423 с/оп у **ranger**;
4. практический выбор реализации зависит от профиля нагрузки: для редко изменяемой таблицы с частыми **Lookup** и **Dump** предпочтителен **ranger**, а для сервиса, который должен регулярно применять инкрементальные BGP-обновления, измерения подтверждают преимущество ART по стоимости изменения состояния.

Последний вывод задаёт границу применимости результата. Выделение функции в микросервис и единый gRPC контракт для нескольких продуктов образуют самостоятельный архитектурный слой; шардированный ART снижает CPU-затраты потокового обновления по сравнению с пакетной перестройкой **ranger**.

9.8. Совокупная стоимость потокового профиля нагрузки

Чтобы перевести бенчмарки в эксплуатационно содержательную плоскость для уже выделенного микросервиса, полезно построить простую модель совокупной стоимости. Для поддержания актуальной таблицы **prefix** -> **ASN** ключевым режимом является не полная выгрузка состояния, а регулярное применение инкрементальных BGP-обновлений. Поэтому проигрыш ART в операции **Dump** сам по себе не определяет выбор хранилища: его нужно сопоставлять со стоимостью **Upsert**, **Delete** и **Lookup** в установившемся потоковом профиле нагрузки. Рассмотрим режим, в котором:

- в системе хранится таблица из n префиксов ($n \in \{10^4; 10^5\}$);
- за единицу времени происходит λ инкрементальных обновлений (**Upsert/Delete**);

- клиенты выполняют μ запросов **Lookup** в секунду;
- операция полного **Dump** вызывается с частотой ν в секунду.

Тогда суммарную CPU-нагрузку на хранилище можно оценить как

$$C(\lambda, \mu, \nu) = \lambda \cdot t_{\text{upsert}}(n) + \mu \cdot t_{\text{lookup}}(n) + \nu \cdot t_{\text{dump}}(n),$$

где t_{upsert} , t_{lookup} и t_{dump} – это средние времена соответствующих операций, взятые из табл. 4–7.

Подставим типичные значения для $n = 10^5$:

- ranger: $t_{\text{upsert}} \approx 6,0 \cdot 10^{-1}$ с; $t_{\text{lookup}} \approx 1,5 \cdot 10^{-7}$ с; $t_{\text{dump}} \approx 2,6 \cdot 10^{-4}$ с;
- шард. ART: $t_{\text{upsert}} \approx 1,7 \cdot 10^{-7}$ с; $t_{\text{lookup}} \approx 4,5 \cdot 10^{-7}$ с; $t_{\text{dump}} \approx 1,4 \cdot 10^{-2}$ с.

Сравнение совокупной стоимости для двух представительных профилей нагрузки приведено в табл. 10.

Таблица 10 – Оценка совокупной CPU-стоимости в установившемся режиме при $n = 10^5$ (приблизённо, CPU-с/с = доля одного CPU-ядра на каждую секунду физического времени)

Сценарий	ranger	шард. ART
$\lambda=10, \mu=10^4, \nu=10^{-2}$	$\approx 6,00$ CPU-с/с	$\approx 0,0045$ CPU-с/с
$\lambda=10^3, \mu=10^5, \nu=10^{-2}$	≈ 600 CPU-с/с	$\approx 0,047$ CPU-с/с

Числа в табл. 10 имеют ясную интерпретацию. В обоих случаях ranger требует более чем одной полной CPU-секунды на каждую секунду физического времени уже при умеренной интенсивности обновлений; для второго профиля оценка по CPU-секундам соответствует сотням CPU-ядер только на обработку изменений, что эксплуатационно неприемлемо. ART-вариант, напротив, в обоих случаях остаётся на уровне долей одного ядра, что показывает возможность использовать его как источник состояния с потоковым обновлением.

Более того, в этой модели хорошо видно, что доминирующая статья расходов меняется. Для варианта с полной перестройкой это всегда обновления (независимо от наличия читателей), для потокового варианта – скорее

Lookup и редкие Dump. Это качественно меняет ту сторону системы, на которой имеет смысл проводить дальнейшую оптимизацию.

Если переложить те же цифры в плоскость требований к задержке актуализации, различие становится более выраженным. Пусть допустимая задержка актуализации таблицы $T_{\text{stale}} \leq 1$ с. Чтобы удерживать такой лаг, пакетной модели пришлось бы перестраивать всю таблицу по несколько раз в секунду; при $n = 10^5$ это ~ 600 мс CPU-работы постоянно. Для потоковой модели то же требование достигается за счёт обработки отдельных событий: каждое обновление применяется за единицы микросекунд или быстрее. Эти два варианта находятся в разных эксплуатационных режимах: первый при умеренной интенсивности обновлений не способен удерживать задержку актуализации на уровне одной секунды без непропорционального расхода CPU.

10. Практические ограничения и направления развития

10.1. Ограничения текущей реализации

Несмотря на положительные результаты по линии «микросервис + потоковое обновление», текущая реализация имеет ряд ограничений.

Во-первых, *Lookup* для IPv6 в существующем варианте ART-хранилища недостаточно оптимален. Это связано с выбранным способом *longest prefix match* (перебор по длинам масок) и представляет собой прямое направление для алгоритмической доработки поверх уже принятой архитектуры выделенного сервиса.

Во-вторых, *Dump* остаётся дороже, чем у *ranger*. В прикладном сценарии это допустимо, если полный снимок вызывается сравнительно редко, но при интенсивном использовании полного дампа может понадобиться дополнительный кэшированный слой полного снимка.

В-третьих, потоковая модель пока в основном рассматривается как последовательность *update*-событий из одного источника. Для более сложных промышленных сценариев могут понадобиться расширенные гарантии повторной доставки, дедупликации и диагностики пропущенных событий [12, 11].

10.2. Возможные доработки реализации

Ограничения, выявленные в экспериментальной части, указывают на три группы доработок текущей реализации.

Lookup. Наиболее заметное ограничение относится к операции LPM поверх ART, особенно для IPv6. Текущий вариант проверяет последовательность кандидатных масок, поэтому стоимость поиска зависит от числа рассматриваемых длин префикса. Возможная доработка состоит в том, чтобы хранить признак терминальности непосредственно в узлах дерева, сохранять последнее найденное валидное совпадение во время прохода по ключу и разделять быстрые пути обработки для IPv4 и IPv6. Такой вариант сохраняет выбранную архитектуру хранилища, но уменьшает число обращений к кан-

дидатным ключам. Работы Tree Bitmap [7] и Poptrie [3] показывают близкие приёмы оптимизации поиска наиболее специфичного префикса.

Dumpr. Для редких, но дорогих полных выгрузок логично ввести кэшированный упорядоченный снимок, ленивую материализацию или потоковую выдачу содержимого вместо одного большого ответа. Если сервис начнёт использоваться как поставщик периодических выгрузок, полезны и дифференциальные снимки для конкретных подписчиков.

Наблюдаемость. Расширение телеметрии и интеграция с BMR-наблюдением [23] и публичными коллекторами [19, 21] позволят контролировать потери событий, сравнивать локальную таблицу с независимым внешним источником и оценивать задержку актуализации состояния. Здесь же уместен механизм, аналогичный route flap damping [25], для подавления гиперактивных префиксов.

Шаблон «отдельный сервис с единым API + снимок + поток + шардированное in-методу ядро» переносим за пределы рассмотренной пары продуктов. Он применим в системах сетевой безопасности, аналитике трафика, операторской отчётности и сетевом инвентаре – везде, где нужно держать актуальный маршрутный контекст для принятия решений и не дублировать логику между потребителями.

ЗАКЛЮЧЕНИЕ

Работа посвящена поддержанию актуального соответствия *IP-префикс* $\rightarrow$ *ASN* по данным BGP для продуктов BIFIT MITIGATOR и BIFIT COLLECTOR. Главный результат – не отдельная микрооптимизация LPM, а сочетание архитектурного и алгоритмического уровней: общий микросервис и потоковое in-memory обновление таблицы.

Вынесение функции в микросервис. Логика построения и обновления таблицы перенесена из состава BIFIT COLLECTOR в автономный процесс с единым gRPC API для нескольких потребителей. Это устраняет дублирование между продуктами и фиксирует границу ответственности: сопоставление по данным BGP находится в одной подсистеме, независимо от выбранного варианта хранилища (*ranger* или шардированный ART).

Потоковое обновление in-memory базы. Реализована операционная модель «начальный снимок + поток событий» от узла GoBGP: обновления применяются к одной общей копии состояния по одному событию, с автоматической пересинхронизацией после разрывов. На материале работ по динамике BGP [13, 6, 9] это согласуется с непрерывным характером маршрутного потока и несовместимостью с одной лишь периодической полной перестройкой.

Структура хранения и корректность снимка. После обзора критериев для префиксных структур в главе 3 в качестве технической основы потока выбран и реализован шардированный Adaptive Radix Tree [14] с операциями Replace, Upsert, Delete, Lookup и Dump. Для внешних клиентов доступны GetASNInfo, GetPrefixesASTable и SubscribePrefixASUpdates. Полный снимок строится без перекрывающихся префиксов в выдаче.

Эксперимент и компромиссы. Сравнение на одном стенде двух реализаций одного prefixStore-контракта показало: при $n = 10^5$ шардированный ART уступает ranger по Dump (порядка 50 раз) и по Lookup для IPv4 в 2–3 раза, для IPv6 по Lookup – на порядок хуже из-за перебора длин масок; зато по Upsert преимущество составляет 5–6 порядков, то есть именно потоковое обновление становится эксплуатационно реальным, а пакетная схема при умеренной интенсивности обновлений требует кратных CPU-ядер только на

пересборку и не укладывается в задержку актуализации на уровне одной секунды. Отдельный прогон на $3 \cdot 10^6$ смешанных IPv4/IPv6 записей усиливает этот вывод: ART выполняет **Replace** примерно за 0,622 с против 27,423 с у **ranger**, а **Upsert** – за 156,8 нс против 28,178 с; при этом **Lookup** и **Dump** на этом наборе остаются быстрее у **ranger**.

Сопоставление с задачами работы: главы 1–2 дают теоретическую базу; глава 3 обосновывает выбор хранилища; глава 4 формулирует требования и гипотезу; главы 5–7 описывают архитектуру микросервиса, алгоритмы снимка и потока, реализацию хранилища; главы 8–9 содержат методику и численные оценки.

Рекомендации: как единый источник пары «префикс $\rightarrow$ ASN» в режиме реального времени сервис с шардированным ART предпочтителен; если доминирует редкая полная выгрузка большой таблицы без потока, оправдан **ranger**; в смешанном профиле – гибрид с кэшем для **Dump**.

Перспективы: нативный LPM поверх ART без полного перебора масок (особенно IPv6), кэшированный упорядоченный снимок для **Dump**, интеграция с BMR [23] и коллекторами [19, 21, 1], проверки на данных, близких к промышленным.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Aben Emile. RIS Beaconing - Shining a Light on BGP Dynamics. — 2023. — Access mode: <https://labs.ripe.net/author/emileaben/ris-beaconing-shining-a-light-on-bgp-dynamics/>.
- [2] Carbone Paris, Katsifodimos Asterios, Ewen Stephan, Markl Volker, Haridi Seif, and Tzoumas Kostas. Apache Flink: Stream and Batch Processing in a Single Engine // Bulletin of the IEEE Computer Society Technical Committee on Data Engineering. — 2015. — Vol. 36, no. 4. — P. 28–38. — Access mode: <https://asterios.katsifodimos.com/assets/publications/flink-deb.pdf>.
- [3] Asai Hirochika and Ohara Yasuhiro. Poptrie: A Compressed Trie with Population Count for Fast and Scalable Software IP Routing Table Lookup // Proceedings of ACM SIGCOMM. — 2015. — Access mode: <https://conferences.sigcomm.org/sigcomm/2015/pdf/papers/p57.pdf>.
- [4] Chen Enke. Route Refresh Capability for BGP-4. — RFC 2918. — 2000. — Access mode: <https://www.rfc-editor.org/rfc/rfc2918.html>.
- [5] Akidau Tyler, Bradshaw Robert, Chambers Craig, Chernyak Slava, Fernández-Moctezuma Rafael J., Lax Reuven, McVeety Sam, Mills Daniel, Perry Frances, Schmidt Eric, and Whittle Sam. The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing // Proceedings of the VLDB Endowment. — 2015. — Vol. 8, no. 12. — P. 1792–1803. — Access mode: <https://vldb.org/pvldb/vol8/p1792-Akidau.pdf>.
- [6] Labovitz Craig, Ahuja Abha, Bose Abhijit, and Jahanian Farnam. Delayed Internet Routing Convergence // Proceedings of ACM SIGCOMM. — 2000. — P. 175–187. — Access mode: <https://>

[//conferences.sigcomm.org/sigcomm/2000/conf/paper/sigcomm2000-5-2.pdf](https://conferences.sigcomm.org/sigcomm/2000/conf/paper/sigcomm2000-5-2.pdf).

- [7] Eatherton Will, Varghese George, and Dittia Zubin. Tree Bitmap: Hardware/Software IP Lookups with Incremental Updates // ACM SIGCOMM Computer Communication Review. — 2004. — Vol. 34, no. 2. — P. 97–122. — Access mode: <https://cseweb.ucsd.edu/~varghese/PAPERS/willpaper.pdf>.
- [8] Fuller Vince and Li Tony. Classless Inter-domain Routing (CIDR): The Internet Address Assignment and Aggregation Plan. — RFC 4632. — 2006. — Access mode: <https://www.rfc-editor.org/rfc/rfc4632.html>.
- [9] Griffin Timothy G., Shepherd F. Bruce, and Wilfong Gordon. The Stable Paths Problem and Interdomain Routing // IEEE/ACM Transactions on Networking. — 2002. — Vol. 10, no. 2. — P. 232–243. — Access mode: <https://www.cs.princeton.edu/~jrex/teaching/spring2005/reading/griffin02.pdf>.
- [10] Gupta Pankaj, Lin Steven, and McKeown Nick. Routing Lookups in Hardware at Memory Access Speeds // Proceedings of IEEE INFOCOM. — 1998. — Vol. 3. — P. 1240–1247.
- [11] Kleppmann Martin. Designing Data-Intensive Applications. — O'Reilly Media, 2017.
- [12] Kreps Jay, Narkhede Neha, and Rao Jun. Kafka: A Distributed Messaging System for Log Processing // Proceedings of NetDB. — 2011. — Access mode: <https://cs.uwaterloo.ca/~ssalihog/courses/papers/netdb11-final12.pdf>.
- [13] Labovitz Craig, Malan G. Robert, and Jahanian Farnam. Internet Routing Instability // Proceedings of ACM SIGCOMM. — 1997. — P. 115–126. — Access mode: <https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/lmj97.pdf>.

- [14] Leis Viktor, Kemper Alfons, and Neumann Thomas. The Adaptive Radix Tree: ARTful Indexing for Main-Memory Databases // Proceedings of IEEE ICDE. — 2013. — P. 38–49. — Access mode: <https://db.in.tum.de/~leis/papers/ART.pdf>.
- [15] Morrison Donald R. PATRICIA - Practical Algorithm To Retrieve Information Coded in Alphanumeric // Journal of the ACM. — 1968. — Vol. 15, no. 4. — P. 514–534. — DOI 10.1145/321479.321481.
- [16] Bates Tony, Chandra Ravi, Katz Dave, and Rekhter Yakov. Multiprotocol Extensions for BGP-4. — RFC 4760. — 2007. — Access mode: <https://www.rfc-editor.org/rfc/rfc4760.html>.
- [17] Nilsson Stefan and Karlsson Gunnar. IP-address Lookup Using LC-tries // IEEE Journal on Selected Areas in Communications. — 1999. — Vol. 17, no. 6. — P. 1083–1092. — Краткая версия в Dr. Dobb's Journal (авг. 1998); основная публикация — эта (IEEE JSAC). Access mode: <https://doi.org/10.1109/49.772439>.
- [18] Patel Keyur, Chen Enke, and Venkatachalapathy Balaji. Enhanced Route Refresh Capability for BGP-4. — RFC 7313. — 2014. — Access mode: <https://www.rfc-editor.org/rfc/rfc7313.html>.
- [19] RIPE NCC. Routing Information Service (RIS). — 2025. — Access mode: <https://www.ripe.net/analyse/internet-measurements/routing-information-service-ris>.
- [20] Rekhter Yakov, Li Tony, and Hares Susan. A Border Gateway Protocol 4 (BGP-4). — RFC 4271. — 2006. — Access mode: <https://www.rfc-editor.org/rfc/rfc4271.html>.
- [21] RouteViews Project. RouteViews API Documentation. — 2025. — Access mode: <https://api.routeviews.org/docs/>.
- [22] Waldvogel Marcel, Varghese George, Turner Jonathan S., and Plattner Bernhard. Scalable High Speed IP Routing Lookups // Proceedings

of ACM SIGCOMM. — 1997. — P. 25–36. — Access mode: <https://kops.uni-konstanz.de/server/api/core/bitstreams/3aed3679-f4db-481c-b132-1775c0e0333a/content>.

- [23] Scudder John, Fernando Rex, and Stuart Stephen. BGP Monitoring Protocol (BMP). — RFC 7854. — 2016. — Access mode: <https://www.rfc-editor.org/rfc/rfc7854.html>.
- [24] Srinivasan Venkatachary and Varghese George. Fast Address Lookups Using Controlled Prefix Expansion // ACM Transactions on Computer Systems. — 1999. — Vol. 17, no. 1. — P. 1–40. — DOI 10.1145/296502.296503.
- [25] Villamizar Curtis, Chandra Ravi, and Govindan Ramesh. BGP Route Flap Damping. — RFC 2439. — 1998. — Access mode: <https://www.rfc-editor.org/rfc/rfc2439.html>.

СПИСОК ІЛЮСТРАТИВНОГО МАТЕРІАЛА

Рисунки

1.	Общая архитектура: сервис между потребителями и узлом GoBGP	33
2.	Внутренние логические слои сервиса	34
3.	Контракт <code>prefixStore</code> и две реализации	36
4.	Конечный автомат жизненного цикла повторного подключения и ресинхронизации	40
5.	Устройство шардированного ART-хранилища	42

Таблицы

1.	Качественное сравнение структур хранения по ключевым критериям	29
2.	Контракт <code>prefixStore</code> и принципиальное поведение реализаций	46
3.	Конфигурация стенда экспериментов	49
4.	Бенчмарк <code>Dump</code> для IPv4 /24	52
5.	Бенчмарк одиночного <code>Upsert</code> поверх заполненной таблицы (IPv4)	53
6.	Бенчмарк <code>Lookup</code> (IPv4 /32 и IPv6 /128, $n \approx 10^5$)	54
7.	Измерение при разном числе шардов: <code>Dump</code> ART, IPv4, без сортировки	54
8.	Сравнение <code>ranger</code> и шардированного ART на смешанном наборе из $3 \cdot 10^6$ записей	55
9.	Итоговая сводка: какой вариант хранилища выигрывает в каком сценарии	56
10.	Оценка совокупной CPU-стоимости в установившемся режиме при $n = 10^5$ (приблизённо, CPU-с/с = доля одного CPU-ядра на каждую секунду физического времени)	58